

14. 메모리 컨트롤러 아키텍처와 RAS — 정독용 심층 해설서

🔥 SK하이닉스 메모리 설계 직무 JD에는 "DRAM Memory Controller 설계 및 분석", "Media Controller(NAND, DRAM)", "RAS 요구사항 분석 및 정책 — HW IP 혹은 FW", "Memory 및 System Level RAS 정책", "Clock/Reset & Power Control Logic"이 반복적으로 등장한다. 그런데 02(DRAM 디바이스)·08(저전력·신뢰성)·09(PHY)에서 본 것은 대부분 "메모리 칩 안쪽의 물리와 회로"였다. 이 문서는 그 칩을 **바깥에서 부리는 두뇌** — 메모리 컨트롤러 — 로 시점을 옮긴다. 🧠 지원자 좌표: 삼성 S.LSI에서 로직 RTL 설계·검증·sign-off를 4년 한 사람. 메모리 컨트롤러는 메모리 시스템에서 **가장 RTL스러운 블록**이다 — FSM, 스케줄러, arbiter, 카운터, 프로토콜 체커. 셀은 새로 배워야 할 갭이지만, 컨트롤러는 **이미 내가 매일 짜고 검증하던 종류의 디지털 로직**이다. 이 문서의 목적은 그 사실을 1st-principles로 끝까지 납득시키는 것이다.

0. 이 문서를 읽는 자세 — 한 문장의 좌표

DRAM 셀(02), NAND 셀(03), PHY(09)를 공부하면 "메모리는 아날로그·소자·공정의 세계"라는 인상이 강해진다. 그러나 그 모든 칩은 **혼자서는 아무 일도 못 한다**. 02에서 본 것처럼 DRAM 셀은 "가난한 전하의 집"이고, 누가 정확한 순서와 정확한 타이밍으로 ACT→RD→PRE를 발행해주지 않으면 데이터를 내놓지도, 지키지도 못한다. 그 "정확한 순서와 타이밍을 만들어내는 주체"가 바로 메모리 컨트롤러다.

이 문서를 관통하는 한 문장은 이것이다.

"셀은 멍청하고, 똑똑함은 전부 컨트롤러에 있다."

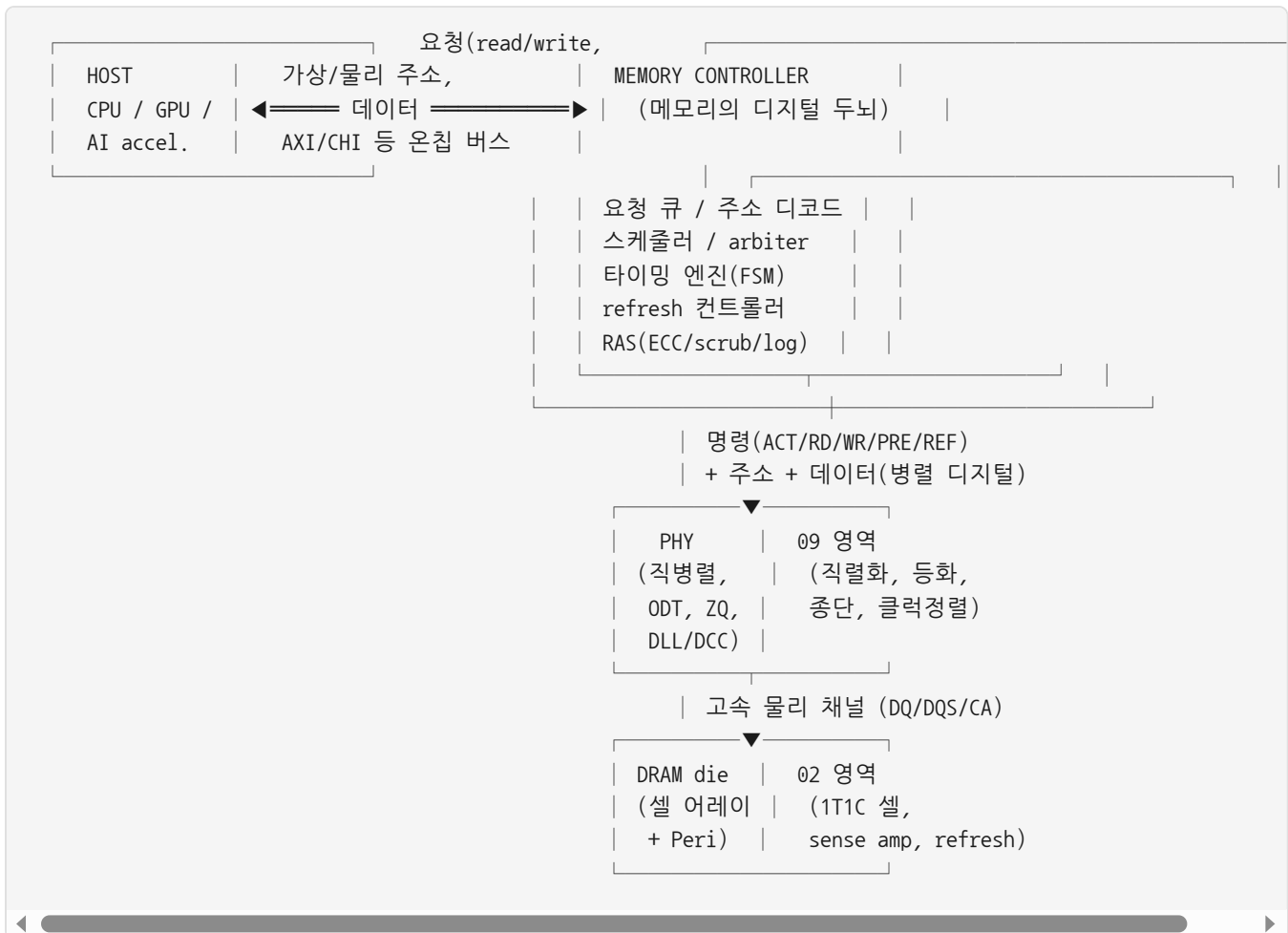
DRAM 칩 자신은 명령을 받으면 시키는 대로 행을 열고 닫을 뿐이다. **무엇을 언제 어느 bank에 발행할지, 타이밍 규칙을 어기지 않으려면 얼마나 기다려야 할지, refresh를 언제 끼워 넣을지, 충돌하는 요청 중 무엇을 먼저 처리할지, 에러가 나면 어떻게 정정·기록·복구할지** — 이 모든 판단이 컨트롤러의 디지털 로직 안에 있다. 그래서 메모리의 성능(대역폭·레이턴시)도, 신뢰성(RAS)도, 전력(저전력 모드 전환)도 결국 컨트롤러가 좌우한다.

읽는 순서: §1(컨트롤러는 어디 있나) → §2(명령/주소 경로) → §3(타이밍 제약 집행) → §4(스케줄러) → §5(refresh 관리) → §6(주소 매핑) → §7(컨트롤러-PHY 경계) → §8(RAS 완전이해) → §9(NAND/HBM 변주) → §10(검증 관점) → §11(면접 골격) → §12(스스로 점검). 각 절은 앞 절이 만든 "왜?"에 답하며 쌓인다.

1. 도입 — 메모리 컨트롤러는 시스템 어디에 앉아 있는가

1.1 호스트 ↔ 컨트롤러 ↔ PHY ↔ DRAM die

먼저 큰 그림을 한 장으로 박아두자. CPU/GPU가 메모리에 데이터를 요청해서 받기까지의 경로다.



이 사슬에서 각자의 역할은 명확히 나뉜다.

- **호스트(CPU/GPU/AI 가속기):** "이 주소를 읽어줘 / 이 주소에 이 데이터를 써줘"라는 **추상적 요청**을 낸다. 호스트는 DRAM의 행/열/bank가 무엇인지, tRCD가 몇 ns인지 전혀 모른다 — 알 필요도 없다.
- **메모리 컨트롤러:** 호스트의 추상 요청을 받아 **DRAM이 알아듣는 구체적 명령 시퀀스(ACT/RD/WR/PRE/REF)와 타이밍**으로 번역한다. 이것이 컨트롤러의 본질적 임무다 — **번역가이자 교통경찰**.
- **PHY(09):** 컨트롤러가 만든 명령·주소·데이터의 **병렬 디지털 비트**를, 고속 물리 채널을 통과할 수 있는 **깨끗한 아날로그 신호**로 송수신한다. 직렬화·등화·종단(ODT)·클럭 정렬(DLL/DCC)이 여기 산다.
- **DRAM die(02):** 명령을 받아 셀 어레이를 활성화·읽기·쓰기·refresh한다. 시키는 대로 할 뿐 스스로 판단하지 않는다(self-refresh 모드의 내부 카운터 정도가 예외 — 그조차 컨트롤러가 "이제 self-refresh로 들어가라"고 명령해야 시작된다).

1.2 컨트롤러는 어디에 물리적으로 존재하나

면접에서 의외로 헛갈리기 쉬운 지점이다. 메모리 컨트롤러는 보통 **메모리 칩 안이 아니라 호스트 쪽(SoC)에 있다**.

- **전통적 DDR 시스템:** 메모리 컨트롤러는 **CPU SoC 내부**의 IP 블록이다(예전엔 노스브리지 칩셋에 있었으나 지금은 CPU 다이에 통합). PHY도 SoC 쪽에 있고, 채널 건너편의 DDR DRAM 칩에는 컨트롤러가 없다.
- **HBM 시스템:** HBM은 DRAM die를 여러 단 쌓고 맨 아래 **base die(logic die)**가 인터페이스를 담당한다(04·09 참조). HBM의 컨트롤러 기능은 보통 **GPU/가속기 SoC 안의 HBM 컨트롤러 + HBM base die의 로직**이 협력해 구현한다. 특히 차세대 HBM에서 base die가 점점 더 똑똑해지며(custom logic), 컨트롤러

·RAS 로직의 일부가 base die로 내려갈 수 있다 — 이것이 SK하이닉스 JD의 "HBM SoC/RTL Design"이 가리키는 영역이다.

- **NAND SSD**: 여기서 컨트롤러가 칩 한복판의 주연이다. NAND 패키지 옆에 별도의 **SSD 컨트롤러 칩**이 붙어 FTL·GC·wear-leveling·ECC를 돌린다(§9, 03 참조). JD의 "Media Controller(NAND, DRAM)"가 이쪽.

지원자 좌표를 미리 박자: **컨트롤러가 어디에 물리적으로 있든, 그것은 RTL로 짜고 검증하는 디지털 로직 IP다.** SoC 안에 있던 HBM base die에 있던 SSD 컨트롤러에 있든, "요청을 받아 FSM으로 명령을 만들고 타이밍을 지키고 에러를 다루는" 그 일의 본질은 같다. 그게 정확히 내가 삼성에서 하던 종류의 일이다.

1.3 왜 "디지털 두뇌"라는 표현이 정확한가

02의 §8에서 DRAM 칩 안의 "Peri(주변회로)"가 디지털 설계의 영역이라고 했다. 컨트롤러는 그 Peri보다 한 단계 더 **순수하게 디지털**이다. Peri에는 sense amp 제어·charge pump 같은 혼성신호가 섞여 있지만, 컨트롤러는 거의 전부가 **레지스터·FSM·카운터·arbiter·큐**로 이루어진다. 아날로그 경계는 PHY가 흡수해주고, 컨트롤러는 그 위에서 깨끗한 디지털 프로토콜만 다룬다.

그래서 컨트롤러는 메모리 시스템에서 **가장 "로직 SoC스러운" 블록**이다. 삼성에서 짜던 AXI 인터커넥트, 캐시 컨트롤러, DMA 엔진과 본질이 똑같다 — 요청을 받아 큐에 쌓고, 우선순위를 정하고, 자원 충돌을 중재하고, 프로토콜을 지켜 하위 디바이스에 명령을 내리는 일. 이 직관을 문서 전체에서 계속 확인할 것이다.

2. 명령/주소 경로 — 추상 요청을 JEDEC 명령으로 번역하기

2.1 호스트 요청이 들어오는 모습

호스트는 컨트롤러에게 대략 이런 형태의 요청을 보낸다(온칩 버스 — AXI/CHI 등 — 위에서).

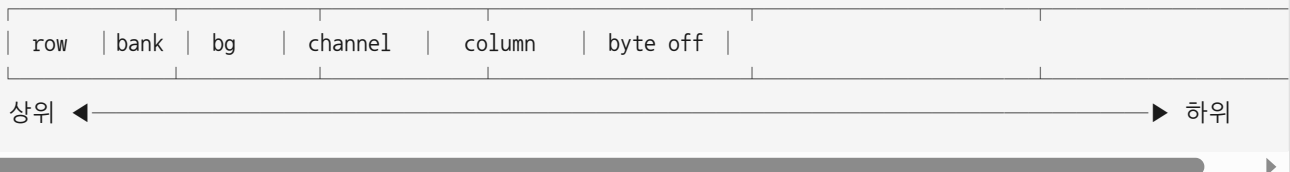
```
{ 동작: READ, 주소: 0x4A3F_8800, 길이: 64바이트(캐시라인), ID: 17 }
{ 동작: WRITE, 주소: 0x4A3F_8840, 데이터: ..., ID: 18 }
```

여기서 주소는 **flat한 물리 주소** 한 덩어리다. 호스트는 이 주소가 DRAM의 몇 번 channel, 몇 번 bank, 몇 번 row, 몇 번 column에 해당하는지 모른다. 컨트롤러가 그것을 풀어내야 한다.

2.2 주소 디코딩 — flat 주소를 (channel/bank/row/column)으로 쪼개기

컨트롤러의 첫 일은 flat 물리 주소를 DRAM의 물리 좌표로 분해하는 것이다. 02에서 본 대로 DRAM 셀 하나는 (channel, rank, bank group, bank, row, column)의 좌표로 지정된다. 컨트롤러는 주소 비트를 잘라(slice) 각 필드에 매핑한다.

물리 주소 비트 (예시, 실제 분할은 구성·정책 의존 — §6에서 깊게)



어느 주소 비트를 어느 필드에 줄 것인가가 곧 §6의 "주소 매핑(address mapping)"이고, 이것이 성능을 좌우하는 핵심 설계 결정이다. 일단 여기서는 "컨트롤러가 주소를 좌표로 푼다"는 사실만 잡자.

2.3 command 변환 — DRAM은 "행을 열고 → 열을 읽고 → 행을 닫는다"

이제 핵심이다. 호스트의 추상 READ/WRITE 하나가, DRAM에게는 여러 개의 저수준 명령 시퀀스로 번역된다. 왜? 02에서 본 DRAM의 물리 때문이다. DRAM은 임의 셀을 바로 못 읽는다. 반드시 **행을 먼저 열고(activate) → 그 행에서 열을 선택해 읽거나 쓰고(read/write) → 다 쓰면 행을 닫는다(precharge)**.

JEDEC이 정의한 주요 명령은 다음과 같다.

명령	약자	하는 일	02 연결
ACTIVATE	ACT	지정 bank의 지정 row를 연다(WL on → charge sharing → sense amp가 row buffer에 올림)	02 §3 읽기 시퀀스의 ①②③
READ	RD	열린 row에서 지정 column을 읽어 DQ로 내보냄	02 §3.4, CL 후 데이터 출력
WRITE	WR	열린 row의 지정 column에 외부 데이터를 씀	02 §4 쓰기 동작
PRECHARGE	PRE	열린 row를 닫고 BL을 VDD/2로 복원	02 §5.3 tRP
REFRESH	REF	다음 차례 row(들)를 read-restore로 되살림	02 §6 refresh
(모드/제어)	MRS, ZQ, SRE/SRX 등	모드 레지스터 설정, ZQ 캘리브레이션, self-refresh 진입/탈출	09 §3.4, 08 §5

그래서 호스트의 READ 하나는 상황에 따라 이렇게 번역된다.

닫혀 있던 row를 읽을 때 (row miss):

ACT(bank b, row r) —tRCD 대기—▶ RD(col c) —CL 대기—▶ data
→ 나중에 필요 없다면 PRE(bank b)

이미 열려 있는 row를 읽을 때 (row hit):

RD(col c) —CL 대기—▶ data ← ACT/PRE 생략, 훨씬 빠름

같은 호스트 요청이라도, 대상 row가 이미 열려 있느냐(row hit) 닫혀 있느냐(row miss/conflict)에 따라 명령 개수와 지연이 완전히 달라진다. 이 차이를 활용·관리하는 것이 §4 스케줄러의 핵심이다.

2.4 JEDEC 프로토콜을 지키는 FSM

컨트롤러의 명령 발행 로직은 본질적으로 **유한상태기계(FSM)**다. 각 bank는 "idle(단침) → activating → active(열림) → reading/writing → precharging → idle"의 상태를 돈다. 컨트롤러는 bank마다 이 상태를 추적하고, 현재 상태에서 합법적인 명령만, 타이밍 규칙(§3)을 만족할 때만 발행한다.

per-bank FSM (개념)



이 FSM이 JEDEC 표준 상태도와 정확히 일치해야 한다. 잘못된 상태 전이(예: ACT 없이 RD, 또는 PRE 안 하고 다른 row를 ACT)는 곧 functional fail이다. JEDEC 프로토콜 준수를 FSM과 assertion으로 보장하는 것 — 이것이 §10에서 다룰 컨트롤러 검증의 핵심이고, 정확히 지원자가 SVA/formal로 하던 일이다.

설계 브릿지: "추상 트랜잭션을 받아 → 프로토콜 상태기계를 돌려 → 하위 디바이스에 합법적 명령 시퀀스를 발행"하는 것은 AXI master, 캐시 컨트롤러, DMA 엔진과 똑같은 패턴이다. DRAM 명령(ACT/RD/PRE)이 AXI의 AR/R/AW/W 채널 핸드셰이크 자리에 들어갈 뿐, FSM·핸드셰이크·프로토콜 준수라는 골격은 동일하다.

3. 타이밍 제약 집행 — 컨트롤러가 지키는 규칙집합

3.1 타이밍 파라미터는 "컨트롤러가 어기면 안 되는 규칙"이다

02 §5에서 tRCD/tRP/tRAS/tRC/tWR 등을 "무슨 물리가 끝나길 기다리는 시간"으로 배웠다. 그 시각은 **DRAM 디바이스 관점**이었다 — "sense amp가 안정될 때까지", "restore가 끝날 때까지". 이제 **컨트롤러 관점**으로 같은 파라미터를 다시 본다.

02 §5의 타이밍 파라미터 = 컨트롤러가 명령을 발행할 때 위반 없이 지켜야 하는 최소 간격의 집합.

DRAM은 명령을 받으면 그냥 실행한다 — 타이밍이 충분히 벌어졌는지 스스로 검사하지 않는다(검사할 뚝뚝함이 없다, §0). 그래서 "tRCD가 안 지나갔는데 RD를 보내면 안 된다"는 규칙을 지키는 책임이 100% 컨트롤러에 있다. 컨트롤러가 규칙을 어기면 DRAM은 불안정한 중간 전압을 읽어 오판하고, 데이터가 깨진다.

3.2 컨트롤러가 추적하는 주요 타이밍 (02 §5 재방문, 컨트롤러 시점)

각 파라미터를 "컨트롤러가 어떤 두 명령 사이에 강제하는 최소 간격인가"로 정리한다.

파라미터	컨트롤러가 강제하는 간격	02의 물리적 이유
tRCD	ACT → (같은 bank의) RD/WR 최소 간격	sense amp가 row를 안정시킬 시간
tRP	PRE → (같은 bank의) 다음 ACT 최소 간격	BL을 VDD/2로 다시 평형시킬 시간
tRAS	ACT → (같은 bank의) PRE 최소 간격(하한)	restore가 끝날 시간 보장
tRC	ACT → (같은 bank의) 다음 ACT (= tRAS+tRP)	한 row 사이클 전체
tWR	마지막 WR 데이터 → (같은 bank) PRE 최소 간격	쓴 값이 셀에 정착할 시간
tCCD_S/L	RD→RD 또는 WR→WR 연속 column 접근 간격(다른/같은 BG)	공유 회로 회복(02 §7.3)
tRRD	ACT → (다른 bank의) ACT 최소 간격	동시 활성화 시 피크 전류 제한
tFAW	임의 4개 ACT가 들어갈 수 있는 최소 윈도우	순간 전력(활성화 전류) 상한
tWTR	WR → RD 전환 최소 간격	내부 데이터 경로 방향 전환
tRFC	REF → 다음 유효 명령까지 bank가 묶이는 시간	refresh가 row를 도는 시간(02 §6)

여기서 02에 없던 두 가지(tRRD, tFAW)를 음미하자. 둘 다 **전력 제약에서 나온 타이밍**이다. ACTIVATE는 WL을 VPP까지 부스트하고 charge pump를 돌리는, DRAM에서 가장 전류를 많이 먹는 동작이다(02 §8). 여러 bank를 너무 빠르게 연달아 activate하면 순간 전류(피크 전류)가 칩 전원망이 감당 못 할 만큼 치솟는다. 그래서 JEDEC은 "ACT끼리 최소 tRRD 띄워라(tRRD)", "어떤 윈도우 안에도 ACT는 4개까지(tFAW)"로 활성화 속도에 상한을 건다. 컨트롤러는 bank 병렬성을 최대한 쓰고 싶지만, 이 전력 타이밍이 발목을 잡는다 — 성능과 전력의 충돌이 타이밍 규칙으로 박혀 있는 것이다.

3.3 컨트롤러는 이것 어떻게 구현하나 — 타이밍 카운터 매트릭스

개념적으로 컨트롤러는 "각 명령이 발행된 시각"을 기억하고, "다음 명령이 합법이 되는 가장 이른 시각"을 계속 계산한다. 보통 bank마다, 그리고 전역으로 작은 down-counter들의 배열을 둔다.

```
bank b: ACT 발행 시각에 카운터 세팅
- tRCD 카운터 → 0이 되면 RD/WR 허용
- tRAS 카운터 → 0이 되면 PRE 허용
전역:
- tFAW 슬라이딩 윈도우(최근 4 ACT의 시각)
- tRRD 카운터(마지막 ACT 이후)
- read/write 버스 방향 전환 카운터(tWTR/tRTW)
```

매 사이클, 스케줄러는 "지금 발행 가능한(모든 관련 카운터=0) 명령"의 후보 집합을 만들고, 그 중 하나를 §4 정책으로 고른다.

즉 컨트롤러의 명령 발행은 "타이밍 게이트를 통과한 후보 중에서 정책으로 한 명령을 고르는" 두 단계다. 타이밍 게이트(§3)가 합법성을, 정책(§4)이 최적성을 담당한다.

설계 브릿지(STA와의 동형): 02 §5.7에서 "타이밍 파라미터 = 메모리판 STA 제약"이라고 했다. 컨트롤러 시점에서 더 정확히 말하면 — STA가 "데이터가 안정되기 전에 클럭이 잡으면 안 된다(setup)"를 정적으로 보장한다면, 컨트롤러의 타이밍 카운터는 "데이터(셀 상태)가 안정되기 전에 다음 명령을 발행하면 안 된다"를

런타임에 동적으로 보장한다. 같은 "안정될 때까지 기다림"의 정적/동적 두 얼굴이다. 그리고 이 카운터 로직 자체는 내가 STA로 닫고 검증하던 RTL이다.

4. 스케줄러 — 왜 명령 순서가 대역폭과 레이턴시를 좌우하나

4.1 문제의식: 요청을 들어온 순서대로 처리하면 느리다

호스트는 요청을 마구잡이 순서로 보낸다. 만약 컨트롤러가 그것을 **들어온 순서 그대로(in-order)** 처리하면 어떻게 될까? 예를 들어 요청이 "bank0 row5 읽기 → bank0 row9 읽기 → bank0 row5 읽기"로 들어오면:

in-order 처리:

ACT(b0,r5) —tRCD— RD ——— PRE —tRP— ACT(b0,r9) —tRCD— RD ——— PRE —tRP— ACT(b0,r5) —tRCD— RD
→ 매번 row를 닫고 다른 row를 여느라 tRC를 통째로 떠안음. 처참한 throughput.

그런데 만약 순서를 바꿔서 "bank0 row5 두 요청을 붙이고, bank0 row9를 나중에" 처리하면, row5를 한 번만 열어 두 요청을 row hit으로 빠르게 처리할 수 있다. **순서를 바꾸는 것만으로 성능이 크게 변한다.** 이것이 메모리 스케줄링의 출발점이다. 컨트롤러는 요청들을 큐에 쌓아두고, 들어온 순서와 무관하게(out-of-order) 가장 효율적인 순서로 재배열한다.

4.2 open-page vs closed-page 정책 — 지역성에 거느냐, 충돌을 피하느냐

가장 근본적인 정책 선택은 "RD/WR을 마친 뒤 그 row를 열어둘 것이냐, 바로 닫을 것이냐"다.

open-page(open-row) 정책 — 열어둔다. - RD/WR 후에도 row를 active 상태로 유지한다. 다음 요청이 **같은 row**(row hit)면 ACT 없이 RD만으로 빠르게 처리한다. - **베팅**: "방금 접근한 row 근처를 또 접근할 것이다"라는 **공간 지역성(spatial locality)**에 건다. 캐시라인·순차 접근이 많은 CPU 워크로드에서 hit률이 높다. - **이득**: row hit이면 tRCD를 통째로 절약 → 레이턴시·전력 절감. - **손해**: 다음 요청이 **같은 bank의 다른 row**(row conflict)면, 먼저 PRE로 닫고(tRP) 다시 ACT(tRCD)해야 한다 — 닫힌 상태에서 시작하는 것보다 **오�히려 느리다**(PRE 추가). 또 열어둔 동안 그 bank는 다른 row를 못 쓴다.

closed-page 정책 — 바로 닫는다. - RD/WR 직후 자동으로 PRE한다(또는 auto-precharge 명령 RDAP/WRAP을 써서 RD/WR과 PRE를 한 번에). - **베팅**: "다음 접근은 어차피 다른 row일 것이다"라는 **낮은 지역성**에 건다. 무작위·산발적 접근(예: 여러 스레드가 뒤섞인 서버, 그래픽 텍스처)에서 유리. - **이득**: 다음 요청이 어느 row든 항상 "닫힌 상태에서 ACT부터" 시작하므로 **row conflict 페널티가 없다**(일관된 latency). PRE를 미리 숨겨둘 수도 있다. - **손해**: 같은 row를 또 접근해도 row hit 이득을 못 누린다(이미 닫았으니 또 ACT).

같은 row 재접근(지역성 높음)

다른 row 접근(지역성 낮음)

open-page: RD (빠름!)	open-page: PRE+ACT+RD
row hit 이득	conflict 페널티
closed: ACT+RD (느림)	closed: ACT+RD (일관됨)

open이 유리

closed가 유리

어느 정책이 옳은가? 워크로드 의존이다. 그래서 실제 컨트롤러는 고정하지 않고 **적응형(adaptive)** 정책을 쓰기도 한다 — row buffer hit률을 모니터링하다가, hit이 잦으면 open으로, conflict가 잦으면 closed로 기운다. 또는 "일정 시간 hit이 없으면 그제야 닫는(timeout-based)" 절충도 흔하다. **"열어둘지 닫을지를 런타임 통계로 판단하는 정책 로직"**이 곧 디지털 제어 블록이다.

4.3 FR-FCFS — 준비된 요청을 먼저

스케줄러의 고전적 알고리즘이 **FR-FCFS(First-Ready, First-Come-First-Served)**다. 이름이 곧 우선순위 규칙이다.

1. **First-Ready (1순위)**: 지금 즉시 발행 가능한(타이밍 게이트 §3를 통과하고, 특히 row hit이라 ACT 없이 바로 갈 수 있는) 요청을 먼저 처리한다.
2. **First-Come-First-Served (2순위)**: 준비도가 같으면, **먼저 들어온** 요청을 먼저 처리한다(기아 방지·공정성).

직관: "이미 열려 있는 row를 때리는 요청(row hit)"은 지금 당장 빠르게 처리되니 먼저 보내고, "닫힌 row를 열어야 하는 요청(row miss)"은 잠시 미룬다. 이렇게 하면 **열린 row를 최대한 우려먹어(row hit을 몰아서) bank 효율을 끌어올린다**.

큐에 쌓인 요청들 (★ = 현재 열린 row를 때림 = ready)
 [req A: b0 r5 ★] [req B: b1 r2] [req C: b0 r5 ★] [req D: b0 r9]
 FR-FCFS 선택 순서:
 A(ready, 먼저 옴) → C(ready) → ... → 그 다음에야 row 바뀌어야 하는 B/D
 → b0의 열린 row r5를 A,C가 몰아서 hit으로 처리

FR-FCFS의 그림자 — 공정성과 기아. row hit을 무조건 우선하면, 특정 row를 계속 때리는 스트림이 다른 요청을 무한정 굶길 수 있다(starvation). 그래서 실제 컨트롤러는 **나이(age) 기반 상한**(너무 오래 기다린 요청은 강제 승격), **read/write 비율 균형**, **QoS(서비스 품질) 우선순위**(예: 디스플레이·실시간 트래픽 우대) 등을 얹어 FR-FCFS를 보강한다. 즉 실제 스케줄러는 "준비도 × 나이 × 우선순위 × 버스 방향"을 종합하는 **다차원 arbiter**다.

4.4 bank/bank-group 병렬성을 활용한 명령 인터리빙

02 §7에서 bank/bank group이 "독립적으로 행을 열 수 있는 병렬 자원"임을 배웠다. 그 병렬성을 만드는 것은 **DRAM 구조이지만, 활용하는 주체는 컨트롤러 스케줄러**다.

스케줄러는 여러 bank의 명령을 시간축에서 **겹쳐서(interleave)** 발행한다.

bank0: ACT — tRCD — RD ————— PRE — tRP —
 bank1: ACT — tRCD — RD ————— PRE —
 bank2: ACT — tRCD — RD —————
 bank3: ACT — tRCD — RD —
 └— 한 bank가 tRCD/tRC로 묶여 있는 동안 다른 bank를 때려
 데이터 버스를 빈틈없이 채운다 (latency hiding)

핵심은 **데이터 버스(DQ)를 한 순간도 놀리지 않는 것**이다. 한 bank의 긴 지연(tRCD, tRC)을 다른 bank의 데이터 전송으로 메운다. 그리고 02 §7.3의 황금률 — **연속 column 접근은 서로 다른 bank group으로 분산(tCCD_S의 빠른 간격을 쓰려고)** — 도 스케줄러가 실행한다. 같은 BG만 연속으로 때리면 tCCD_L에 걸려 버스에 구멍이 생기므로, 스케줄러는 의도적으로 BG를 번갈아 고른다.

4.5 read/write turnaround — 버스 방향 전환 비용과 grouping

DQ 버스는 **양방향**이다 — 읽기는 DRAM→컨트롤러, 쓰기는 컨트롤러→DRAM. 그런데 같은 물리 선의 방향을 바꾸려면 **공백 시간(bus turnaround)**이 필요하다. 드라이버를 끄고 켜고, ODT(09 §3.3)를 재설정하고, 신호가 정착하기를 기다려야 한다(tWTR: write→read, tRTW: read→write).

방향을 자주 바꾸면 (RWRWRW):

RD—[turnaround]—WR—[turnaround]—RD—[turnaround]—WR ...
→ turnaround 공백이 누적되어 버스 효율 폭락

같은 방향끼리 묶으면 (RRRR...WWW...):

RD RD RD RD —[turnaround 한 번]— WR WR WR WR
→ turnaround 비용을 분산(amortize)

그래서 스케줄러는 읽기는 읽기끼리, 쓰기는 쓰기끼리 묶어(batch) 처리한다. 보통 쓰기는 급하지 않다(호스트는 쓰기를 "보내고 잊는다" — 결과를 기다리지 않음). 반면 읽기는 급하다(호스트가 데이터를 기다리며 멈춰 있다 — 레이턴시 직결). 그래서 컨트롤러는:

- **읽기 우선**: 읽기를 빠르게 처리해 호스트 stall을 줄인다.
- **write buffering(쓰기 버퍼링)**: 쓰기를 큐(write buffer)에 모아뒀다가, 읽기가 잠잠해지는 틈이나 버퍼가 일정 수위(high watermark)를 넘을 때 몰아서 한 방향으로 비운다(write drain).
- **write-to-read forwarding/hazard 처리**: 버퍼에 아직 안 쓴 데이터를 같은 주소가 읽으려 하면, 버퍼에서 직접 데이터를 넘겨주거나(forwarding) 순서를 지켜준다 — **메모리 일관성(consistency)**을 깨지 않기 위한 의존성 검사다.

설계 브릿지: read/write batching, write buffer watermark, hazard forwarding — 이 전부가 캐시-스토어 버퍼-메모리 디스앰비규레이션에서 보던 그 로직이다. "급한 읽기 먼저, 쓰기는 모아서 한가할 때, 단 같은 주소 의존성은 지켜라" — 컴퓨터 구조의 보편 원리이고, 그것을 RTL FSM·큐·비교기로 구현·검증하는 것이 내 일이다.

4.6 왜 스케줄링이 대역폭·레이턴시를 좌우하나 — 종합

정리하면, 같은 DRAM 칩이라도 스케줄러의 품질에 따라 실효 성능이 크게 갈린다.

- **대역폭(throughput)**: row hit률을 높이고(open-page/FR-FCFS), bank·BG 병렬성으로 버스를 빈틈없이 채우고(interleaving), turnaround를 줄이면(batching) — **버스 점유율(bus utilization)**이 올라가 실효 대역폭이 이론 최대에 가까워진다. 스케줄링이 나쁘면 ACT/PRE 오버헤드와 turnaround 공백으로 버스가 절반도 못 차는 일이 흔하다.
- **레이턴시**: 급한 읽기를 우선 처리하고(read priority), row hit으로 ACT를 건너뛰면 평균 응답 시간이 준다. 단 공정성(기아 방지)과 충돌하므로 균형이 필요하다.

핵심 한 문장: DRAM의 이론 대역폭은 셀·핀·클럭이 정하지만, 그 이론을 실제로 얼마나 뽑아내느냐는 컨트롤러 스케줄러가 정한다. 스케줄러는 "타이밍 규칙(§3)을 안 여기면서, 병렬성을 최대로 쓰고, 급한 것을 먼저"를 매 사이클 푸는 최적화 엔진이고, 전형적인 디지털 제어·arbitration 로직이다.

5. Refresh 관리 — 일반 트래픽과 새는 전하의 줄다리기

5.1 컨트롤러는 refresh를 "끼워 넣는다"

02 §6에서 refresh의 물리(누설 → retention → 주기적 read-restore)와 tREFI($\approx 7.8\mu s$)를 배웠다. 그것은 "DRAM이 refresh를 받으면 무슨 일을 하나"였다. 이제 누가 언제 refresh 명령을 발행하나 — 컨트롤러다.

컨트롤러는 refresh 타이머를 돌려, 평균 tREFI마다 REFRESH 명령을 발행한다. 그런데 여기에 근본적 긴장이 있다.

refresh 중인 bank는 일반 read/write를 못 받는다(tRFC 동안 묶인다). 그런데 일반 트래픽도 그 bank를 쓰고 싶어 한다. 둘이 충돌한다.

즉 refresh는 "데이터를 살리기 위해 반드시 해야 하지만, 하는 동안 성능을 깎아먹는" 세금이다(02 §6.4). 컨트롤러의 임무는 이 세금을 언제 낼지 영리하게 스케줄링해서, 데이터는 지키되 성능 충격은 최소화하는 것이다.

5.2 all-bank refresh vs per-bank refresh

REFRESH의 단위가 두 가지다.

- **all-bank refresh (REFab):** 모든 bank를 한꺼번에 refresh한다. 그동안 칩 전체가 일반 접근 불가. 구현은 단순하지만, refresh 시간(tRFC) 내내 그 rank 전체가 멈춘다.
- **per-bank refresh (REFpb, LPDDR/일부 DDR5):** 특정 bank만 refresh하고, 나머지 bank는 계속 접근 가능하게 한다. refresh를 잘게 쪼개 분산하므로, 일반 트래픽과 더 잘 겹친다 — 성능 충격이 작다. 대신 컨트롤러가 "어느 bank를 언제 refresh했는지, 다음 차례는 어디인지"를 추적해야 해 제어가 복잡하다.

```
all-bank:  [——— REF (전 bank 멈춤, tRFCab) ——] 일반 트래픽 대기
per-bank:  b0:[REF] b1:접근 b2:접근 b3:접근
           b1:접근 b0:접근 b1:[REF] ...    ← refresh와 접근을 겹침
```

per-bank refresh는 "한 bank가 refresh로 묶인 동안 다른 bank로 일반 트래픽을 흘려보내" 02 §7의 bank 병렬성을 refresh에도 적용하는 셈이다.

5.3 refresh를 미루기(postpone)와 당기기(pull-in)

JEDEC은 refresh에 유연성을 준다. 매번 정확히 tREFI마다 칠 필요는 없고, 평균만 맞으면 된다. 그래서 컨트롤러는:

- **postpone(미루기):** 지금 트래픽이 폭주 중이면 refresh를 잠시 미룬다. JEDEC은 보통 최대 8회(또는 9회)까지 refresh를 밀어둘 수 있게 허용한다(refresh "빔"을 진다). 트래픽 burst가 지나가면 밀린 refresh를 갚는다.
- **pull-in(당기기):** 반대로 지금 한가하면, 앞으로 칠 refresh를 미리 당겨서 몰아 친다. 그러면 곧 올 트래픽 burst 때 refresh로 방해받지 않는다.

refresh 비트 카운터를 두고:

트래픽 busy → REF postpone (비트 +1, 최대 한도까지)

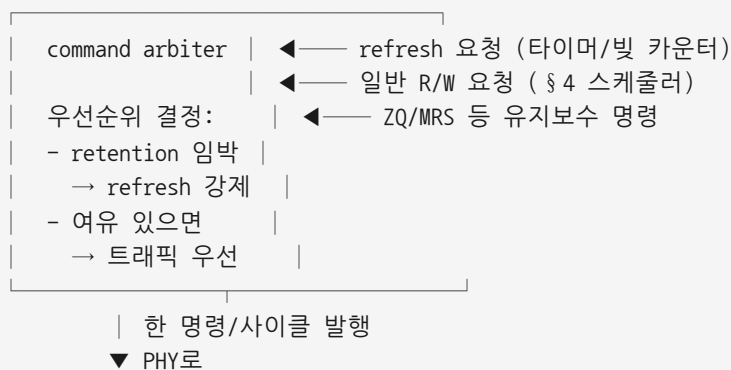
트래픽 idle → REF pull-in (비트 -1, 미래 여유분 선결제)

단, 어떤 순간에도 "retention 한도(64ms 안에 모든 row)"는 절대 위반 금지

핵심: "평균 tREFI는 지키되, 순간순간은 트래픽 상황에 맞춰 refresh를 유연하게 배치"한다. 이것이 refresh가 성능에 주는 충격을 줄이는 컨트롤러의 핵심 기술이고, postpone 한도와 retention 한도라는 **하드 제약 안에서 움직이는 카운터 기반 스케줄링**이다.

5.4 일반 트래픽과의 arbitration

결국 refresh 컨트롤러는 §4의 일반 스케줄러와 **자원(bank, 명령 버스)을 놓고 경쟁**한다. 둘 사이의 중재(arbitration)가 refresh 관리의 본질이다.



arbiter는 "retention 마감에 임박했으면 refresh를 무조건 우선, 여유가 있으면 일반 트래픽 우선"이라는 **마감 기반 우선순위(deadline-aware)**로 동작한다. 이것은 실시간 시스템의 EDF(Earliest Deadline First) 스케줄링과 같은 사고다.

5.5 self-refresh 진입/탈출 제어

08 §5에서 self-refresh를 "DRAM이 스스로 내부 타이머로 refresh를 돌리는 저전력 standby 모드"로 배웠다. 그 **진입과 탈출을 명령하는 것도 컨트롤러**다.

- **진입(SRE, Self-Refresh Entry)**: 일정 시간 트래픽이 없으면(idle 감지), 컨트롤러가 "지금 열린 row를 모두 PRE로 닫고 → SRE 명령 발행 → 외부 클럭을 끌 수 있음"의 시퀀스를 실행한다. 이후 DRAM은 자기 내부 타이머·온도 센서(TCSR)로 refresh를 돌리고, 컨트롤러·PHY는 대부분 잠든다.
- **탈출(SRX, Self-Refresh Exit)**: 새 요청이 오면, 컨트롤러가 클럭을 다시 안정시키고(DLL/PLL 재락, ZQ 재보정이 필요할 수 있음) → SRX 명령 → 정해진 복귀 시간(tXS 등) 대기 → 일반 동작 재개. 이 **wake-up latency** 때문에, self-refresh는 "충분히 오래 idle할 때"만 이득이다(08 §4.2 power gating의 논리와 동형).

설계 브릿지: self-refresh 진입/탈출은 **clock/reset & power control logic** 그 자체다(JD 키워드!). "언제 idle로 판단해 진입할지(타이머), 진입 시 어떤 순서로 row를 닫고 클럭을 끌지(시퀀스 FSM), 탈출 시 클럭·캘리브레이션 어떻게 복원할지(wake 시퀀스)" — 전부 FSM·타이머·시퀀서다. 그리고 self-refresh에서는 내부 저속 클럭으로 전환되므로 **클럭 도메인 경계가 생겨 CDC가 끼고**(08 §5, 05 참조), 복귀 시퀀스의

타이밍은 STA가 본다. 08에서 본 "저전력 모드 전환 = 디지털 제어 로직 + CDC + STA"가 컨트롤러 측에서 구현되는 모습이다.

6. 주소 매핑 — 연속 주소를 흠어 병렬성을 만든다

6.1 주소 매핑이 왜 성능 결정인가

§2.2에서 "flat 물리 주소를 (channel/bank/bg/row/column)으로 쪼갬다"고 했다. 어느 주소 비트를 어느 필드에 배정하느냐가 곧 주소 매핑(address mapping/interleaving)이고, 이것이 §4 스케줄러만큼이나 성능을 좌우한다.

핵심 직관: **호스트의 접근은 보통 연속적이다**(순차 읽기, 캐시라인 스트림, 행렬 순회). 만약 연속된 주소들이 전부 같은 bank의 같은 row로 떨어지면, 그것들을 직렬로 처리해야 해 병렬성을 못 쓴다. 반대로 연속 주소가 서로 다른 bank/channel로 흠어지면, 02 §7의 bank 병렬성과 §4의 interleaving으로 동시에 처리할 수 있다.

그래서 주소 매핑의 목표는 "흔히 함께 오는 연속 주소들을 최대한 다른 bank/BG/channel로 분산시키는 것"이다.

6.2 두 극단의 매핑

접근 패턴: 주소 0,1,2,3,4,5,6,7 ... 을 순차 접근한다고 하자.

[나쁜 매핑] 하위 비트를 column에 몰아줌

0,1,2,... → 전부 같은 bank의 같은 row, 다른 column
→ 한 bank만 죽어라 쓰고 나머지 bank는 논다. 병렬성 0.

[좋은 매핑] 하위 비트 일부를 bank/channel에 배정

0→ch0/b0, 1→ch1/b1, 2→ch0/b2, 3→ch1/b3 ...
→ 연속 주소가 여러 channel · bank로 흠어짐
→ 스케줄러가 동시에 여러 bank를 때려 버스를 가득 채움. 병렬성 최대.

채널 인터리빙은 한 발 더 나간다. 연속 주소를 여러 독립 channel(각자 자기 데이터 버스를 가진)로 흠으면, 채널들이 완전히 병렬로 동작해 실효 대역폭이 채널 수에 비례해 커진다. HBM이 채널을 잘게 쪼개는(다채널 · pseudo-channel, §9) 이유 중 하나가 이 인터리빙 병렬성이다.

6.3 매핑은 워크로드와 정책에 얹힌다

완벽한 단일 매핑은 없다. 트레이드오프가 있다.

- **bank/channel을 하위 비트에 두면(잘게 흠기):** 순차·스트리밍 워크로드에서 병렬성 최대. 단 공간 지역성(같은 row 재접근)은 희생될 수 있어, open-page 정책의 row hit 이득과 충돌할 수 있다.
- **row를 하위에 더 두면(같은 row에 더 머물기):** row hit 지역성은 살지만 병렬성이 준다.

그래서 매핑은 타깃 워크로드(서버/모바일/그래픽/AI)와 §4 페이지 정책을 함께 고려해 설계한다. 일부 시스템은 XOR 해싱으로 bank 비트를 섞어, 특정 stride 접근이 같은 bank에 몰리는 bank conflict 패턴(pathological

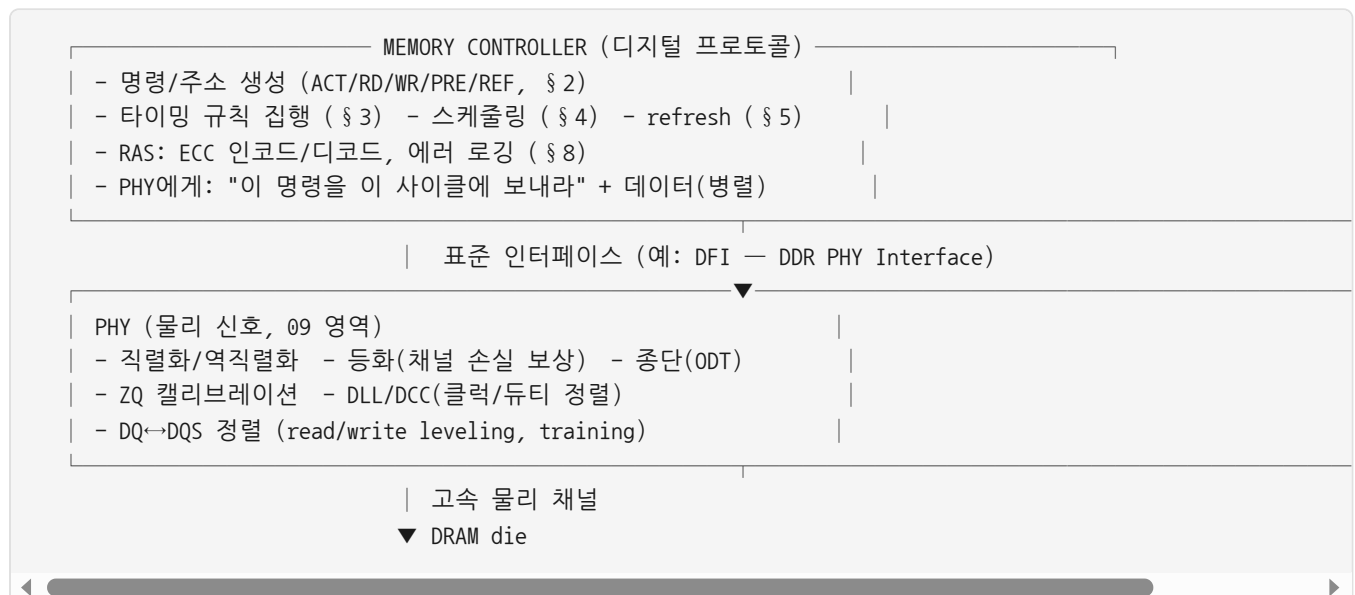
stride)을 깨뜨린다. 핵심은 — 매핑은 "주소 비트를 어떻게 자르고 섞느냐"는 **순수 조합 로직(주소 디코더)**이지만, 그 선택이 시스템 성능을 좌우하는 **아키텍처 결정**이라는 점이다.

설계 브릿지: 주소 비트를 잘라 자원에 매핑하고, 충돌 패턴을 XOR로 흘리는 것은 캐시 set indexing·인터리브드 메모리·뱅크된 SRAM에서 보던 그 기법이다. 그리고 매핑 로직이 바뀌면 §2의 주소 디코드 RTL이 바뀌므로, 그 정확성(모든 주소가 빠짐없이 유일한 좌표로 매핑되는가)을 검증하는 것도 내 일이다.

7. 컨트롤러-PHY 경계 — 디지털 명령을 물리 신호로 넘기기

7.1 경계의 정의 — 누가 무엇을 책임지나

09에서 PHY를 깊게 다뤘다(직렬화, 등화 FFE/CTLE/DFE, 종단 ODT, ZQ, DLL/DCC, 아이 다이어그램). 여기서는 컨트롤러와 PHY가 만나는 국경만 본다. 둘의 책임 분담은 대략 이렇다.



둘 사이의 경계는 보통 **표준 인터페이스(DFI, DDR PHY Interface 등)**로 규격화된다. 컨트롤러는 "어느 사이클에 어떤 명령·데이터를 줄지"를 DFI 프로토콜로 PHY에 넘기고, PHY는 그것을 물리 채널로 송수신한다. 이 표준화 덕에 컨트롤러 IP와 PHY IP를 **분리 개발·재사용**할 수 있다(삼성에서 보던 IP 인터페이스 표준화와 같은 발상).

7.2 컨트롤러가 조율하는 PHY 동작들

PHY의 세부 회로(등화 탭, ODT 임피던스, DLL 락)는 PHY가 알아서 하지만, **언제 그것을 하라고 명령·조율하는 것은 컨트롤러다**.

- **training/leveling 조율(09 §5):** 부팅 시 또는 주기적으로, DQ를 DQS의 정확한 위치에 맞추는 **read/write leveling, DQ deskew, Vref training**을 수행한다. 실제 비트 시프트·샘플링 위치 탐색은 PHY가 하지만, "**지금 training을 시작하라, training 패턴을 보내라, 결과를 모아 최적 설정을 정하라**"는 시퀀스를 컨트롤러(또는 펌웨어)가 지휘한다. 부팅 시퀀스의 한 단계로 들어간다.
- **ODT 제어(09 §3.3):** 읽기/쓰기, 그리고 멀티랭크에서 어느 rank가 active인지에 따라 **ODT를 언제 켜고 끌지**를 컨트롤러가 명령 타이밍에 맞춰 제어한다. ODT enable 타이밍이 틀리면 그 순간 반사로 아이가 닫힌다

(09 §3.3 브릿지) — 즉 ODT 제어 FSM의 타이밍이 곧 SI를 좌우한다.

- **ZQ 캘리브레이션 트리거(09 §3.4)**: 온도가 변하면 임피던스가 틀어지므로, 컨트롤러가 주기적으로 ZQCS/ZQCL 명령을 발행해 PHY/DRAM 임피던스를 재보정시킨다.
- **DQS/데이터 타이밍 관리**: 소스 동기 방식(09 §5.1)이라 DQ는 DQS와 함께 움직인다. 컨트롤러는 write 시 DQS를 데이터 중앙(write는 DQS가 DQ 중앙), read 시 PHY가 DQS를 edge로 받아 중앙으로 시프트하는 등의 정렬을, training 결과에 따라 설정한다.

7.3 clock/reset & power domain 제어 로직 (JD 키워드)

JD에 명시된 "Clock/Reset & Power Control Logic"은 컨트롤러-PHY 경계에서 특히 두드러진다. 메모리 서브시스템은 여러 클럭 도메인과 전원 도메인으로 나뉜다.

도메인 예시:

- 코어/컨트롤러 클럭 (호스트 버스 주파수)
- 메모리 클럭 (DRAM 동작 주파수, 예: speed bin별)
- PHY 내부 클럭 (DLL/PLL이 만든 고속 I/O 클럭)
- self-refresh 시 내부 저속 클럭

전원:

- 컨트롤러 전원, PHY 전원, ODT/termination 전원
- 저전력 모드(self-refresh/power-down)에서 일부 도메인 차단

컨트롤러는 이 도메인들 사이의 **클럭 정렬·리셋 시퀀스·전원 시퀀스**를 제어한다. 부팅 시 "어느 순서로 전원을 올리고, 리셋을 풀고, PLL을 락하고, training을 하고, 정상 동작으로 들어가는가"의 **전원-온 시퀀스(power-on sequencing)**, 저전력 진입 시 "어느 도메인을 어떤 순서로 끄는가", 복귀 시 "어떻게 되살리는가" — 전부 컨트롤러의 시퀀서 FSM이다.

설계 브릿지(이 절이 지원자에게 특히 강력): 클럭 도메인이 여럿이면 그 경계를 넘는 신호는 전부 CDC(05 참조)다 — 컨트롤러 클럭에서 만든 enable이 PHY/메모리 클럭 도메인으로 넘어가고, self-refresh의 저속 클럭으로 전환되는 모든 지점이 CDC 검증 대상이다. 리셋이 도메인마다 비동기로 풀리면 **RDC(Reset Domain Crossing)**다(05 참조). 전원 도메인 차단은 08 §4의 power gating(isolation·retention FF·시퀀싱)이다. 즉 "Clock/Reset & Power Control Logic"은 내가 CDC/RDC sign-off와 UPF power-aware 검증으로 매일 다루던 바로 그 영역이다. 메모리 디바이스를 몰라도, 다중 도메인 제어 로직의 검증은 내 홈그라운드다.

8. RAS 완전이해 — Reliability, Availability, Serviceability

8.1 RAS란 무엇인가 — 세 단어의 정의

RAS는 시스템이 **에러에 직면해서도 신뢰할 수 있게, 멈추지 않게, 고칠 수 있게** 만드는 기능의 총칭이다. 메모리에서 점점 더 중요해지는, 그리고 SK하이닉스 JD가 별도 키워드로 박아둔 영역이다.

- **Reliability(신뢰성)**: 데이터·동작이 **틀리지 않게** 한다. 에러를 검출하고 정정한다(ECC). "잘못된 답을 내지 않는다."
- **Availability(가용성)**: 에러가 나도 시스템이 **멈추지 않고 계속 동작**한다. 일부가 고장나도 degraded mode로 버틴다. "다운타임을 최소화한다."

- **Serviceability(서비스성):** 에러를 진단·기록·복구·교체할 수 있게 한다. 어디서 무슨 에러가 났는지 로깅하고 리포팅해, 운영자가 대응(예: 약한 DIMM 교체)할 수 있게 한다. "무슨 일이 났는지 알고 고칠 수 있다."

세 단어를 한 흐름으로 묶으면: 에러를 정정해 답을 지키고(R), 정정하며 멈추지 않고(A), 무슨 에러였는지 기록해 나중에 고친다(S).

8.2 왜 AI·데이터센터에서 RAS가 필수인가

RAS는 옛날엔 고급 서버의 선택 사양이었다. 지금 — 특히 AI 시대에 — **필수**가 된 이유는 규모와 시간의 곱이다.

- **메모리 총량의 폭증:** AI 서버 한 대에 수백 GB~TB의 HBM/DDR이 들어가고, 그것이 수만 대 모인다. 셀 하나의 에러 확률이 아무리 작아도, **셀 수 × 시간**을 곱하면 "어딘가에서 항상 에러가 나고 있는" 상태가 된다(08 §8.1, 미세화로 raw bit error rate도 오른다).
- **긴 연속 실행:** 대규모 AI 학습은 **수일~수주** 멈추지 않고 돈다. 그 도중 단 한 번의 정정 불가 에러 (uncorrectable error)가 전체 잡(job)을 죽이면, 수천 GPU·수일의 계산이 날아간다 — 막대한 비용. 그래서 "에러가 나도 살아남는" 가용성이 곧 돈이다.
- **silent data corruption(SDC)의 공포:** 검출 못 한 에러가 조용히 잘못된 결과를 만들면, AI 모델이 미묘하게 틀린 답을 내놓아도 아무도 모른다. 그래서 **검출(detect)**이 정정만큼 중요하다 — 못 고치면 적어도 "틀렸다"고 알려야(poison) 잘못된 결과의 전파를 막는다.

즉 AI/데이터센터에서 RAS는 "**있으면 좋은 것**"이 아니라 "**없으면 시스템이 못 굴러가는 것**"이다. 그래서 메모리 컨트롤러·HBM base die의 RAS 로직 설계가 핵심 직무가 된다.

8.3 ECC 계층 — on-die vs link vs system-level

08 §8에서 on-die ECC와 system ECC의 차이를 배웠다. RAS 관점에서 **ECC가 여러 계층으로 쌓여 각자 다른 종류의 에러를 막는다는** 점을 더 명확히 한다.

계층	위치	막는 에러	코드 강도	가시성
on-die ECC	DRAM 칩 내부(02 §8, 08 §8)	셀 어레이의 single-bit error (미세화로 늘어난 것)	약(SECDED급)	호스트에 거의 투명
link/PHY ECC	전송 채널(특히 HBM, 09 §8.4)	채널 전송 중 발생한 비트 에러 (crosstalk·노이즈·jitter 유발)	중	컨트롤러가 인지·재시도
system/rank-level ECC	메모리 컨트롤러/CPU 쪽 (ECC DIMM, chipkill)	다중 칩·다중 비트·칩 단위 fail	강(다중비트 정정, 칩 한 개 죽어도 복구)	호스트가 관리·리포팅

핵심은 **계층적 보완(defense in depth)**이다.

- **on-die ECC**는 칩이 "출하 품질"을 유지하려 내부에서 자잘한 single-bit를 닦는다. 호스트는 모른다.
- **link ECC**는 컨트롤러↔DRAM **전송 경로**에서 생긴 에러를 잡는다. 09에서 본 SI 열화(반사·crosstalk·jitter)가 비트를 뒤집을 수 있는데, link ECC(또는 link CRC + retry)가 이를 검출해 **재전송(retry)**한다. HBM처럼 고속·광폭이라 전송 에러 위험이 큰 경우 특히 중요하다.
- **system ECC**는 위 둘이 못 막는 큰 사고 — 칩 하나가 통째로 죽는 경우(**chipkill/SDDC, Single Device Data Correction**) — 까지 살아남게 하는 가장 큰 그물이다. 여러 칩에 데이터를 분산 인코딩해, 한 칩 전체가 fail해도 나머지로 복원한다(RAID와 같은 발상의 메모리판).

에러가 막히는 계층 (안쪽 → 바깥쪽)
[셀 에러] → on-die ECC가 정정 (대부분 여기서 끝)
| 못 막으면
[전송 에러] → link ECC/CRC가 검출 → retry
| 못 막으면
[칩/다중비트] → system ECC(chipkill)가 정정
| 못 막으면
[정정 불가(UE)] → poison 표시 + 로깅 + (가능하면) 복구, 아니면 정지

주의(08 §8.3과 연결): on-die ECC가 가린 에러는 system 통계에 안 잡혀, 운영자가 "이 DRAM이 사실 약해지고 있다"는 신호를 놓칠 수 있다(계층 간 가시성 단절). 그래서 최신 표준은 on-die ECC가 정정한 에러도 외부에 리포팅하는 메커니즘(예: error check & scrub 관련, 세대별 발전 중)을 둔다.

8.4 scrubbing — 잠자는 비트 오류를 청소한다

scrubbing(스크러빙)은 RAS의 핵심 능동 기법이다. 문제의식: 메모리의 어떤 영역은 **오래 안 읽힐 수 있다**. 그동안 그 자리에 single-bit error가 하나 생기면, 아무도 안 읽으니 정정도 안 된다. 그러다 시간이 더 흘러 두 번째 **비트까지 에러가 나면**, 이제 SECEDED로는 정정 불가(2비트는 검출만)가 된다. 즉 "**고칠 수 있었던 1비트 에러를, 방치해서 고칠 수 없는 2비트 에러로 키우는**" 사고가 난다.

scrubbing은 이를 막는다 — **주기적으로 메모리를 읽어, ECC로 에러를 정정해 다시 써넣는다**. 잠자는 동안 쌓일 뻔한 에러를 미리미리 닦아, 1비트일 때 잡는다.

두 종류가 있다.

- **patrol scrub(순찰 스크럽):** 백그라운드에서 **천천히, 트래픽이 한가할 때** 메모리 전체를 순회하며 읽고 정정·재기록한다. 일반 트래픽에 방해되지 않게 낮은 우선순위로 둔다. 한 바퀴 도는 데 시간(예: 수 시간~하루급, 구성 의존)이 걸리지만, "쌓이기 전에 닦는다"는 예방 효과.
- **demand scrub(요구 스크럽):** 일반 읽기 중 ECC가 정정 가능한 에러를 발견하면, 그 자리를 즉시 정정해 다시 써넣는다. 읽다가 발견한 에러는 그 김에 고친다.

patrol scrub (배경 순찰):
[한가할 때] addr++ 읽기 → ECC 검사 → CE면 정정 후 재기록 → 다음 addr
→ 메모리 전체를 주기적으로 훑어 잠재 에러를 1비트일 때 제거

demand scrub (읽다가 발견):
일반 RD → ECC가 CE 발견 → 그 자리 정정 재기록 (write-back)

scrubbing은 02 §6의 refresh와 비슷하지만 목적이 다르다 — **refresh는 누설로 새는 전하를 되채우는 것(physical)**, **scrubbing은 ECC로 비트 오류를 정정하는 것(logical)**. 둘 다 "주기적으로 메모리를 돌며 데이터를 건강하게 유지"하는 background 작업이고, 둘 다 컨트롤러가 일반 트래픽과 arbitration하며 끼워 넣는다.

8.5 error logging/reporting — CE와 UE를 세고 기록한다

Serviceability의 핵심이다. 컨트롤러는 발생한 에러를 **분류·집계·기록·보고**한다.

- **CE(Correctable Error, 정정 가능 에러):** ECC가 고칠 수 있는 에러(보통 single-bit). 시스템은 정상 동작하지만, **컨트롤러는 이것을 카운트**한다. 특정 주소·DIMM·rank에서 CE가 비정상적으로 자주 나면 → "이

부분이 약해지고 있다"는 조기 경보다. 운영자가 그 DIMM을 **고장 전에 교체(predictive failure)**할 수 있게 한다.

- **UE(Uncorrectable Error, 정정 불가 에러)**: ECC가 검출은 했지만 못 고치는 에러(보통 multi-bit). 이걸 심각하다. 잘못된 데이터를 그냥 쓰면 안 되므로, 컨트롤러는 그 데이터를 **poison(오염 표시)**한다(§8.6).

에러 카운터 · 로그 (per-rank / per-DIMM / per-address)
CE 카운터 → 임계 초과 시 인터럽트/로그 → 예측적 교체 신호
UE 발생 → 즉시 로깅 + poison + (정책에 따라) 머신체크/복구
leaky bucket: CE가 "시간당 몇 회"를 넘으면 경보 (일시적 vs 영구 구분)

이 로깅은 보통 **하드웨어 에러 레지스터 + 펌웨어/OS의 머신체크 아키텍처(MCA 등)**가 협력해 구현한다. JD의 "RAS 정책 — HW IP 혹은 FW"가 바로 이 부담을 가리킨다 — **빠른 검출·정정·즉각 대응은 HW(컨트롤러 RTL)가, 정책 판단·로그 집계·교체 결정은 FW가** 맡는 식의 하드웨어/펌웨어 협력 설계.

8.6 poison과 retry — 못 고치면 격리하고, 일시적이면 재시도

- **poison(오염 전파 차단)**: UE가 난 데이터를 그냥 호스트에 넘기면, 호스트가 그 틀린 값으로 잘못된 계산을 한다(SDC). 그래서 컨트롤러는 UE 데이터에 **poison 마커**를 붙여 "이건 믿지 마라"고 표시한다. 그 데이터를 실제로 **소비(consume)**하는 **시점에서만** 에러를 일으키게 해(예: 그 캐시라인을 실제 쓰는 명령에서 fault), 안 쓰이는 데이터의 에러로 시스템 전체를 죽이지 않는다 — **에러를 격리(containment)**해 가용성을 높인다.
- **retry(재시도)**: 에러가 **일시적(transient)**일 수 있다(전송 노이즈, 잠깐의 글리치). 같은 읽기를 다시 하면 정상일 수 있다. 그래서 link CRC가 전송 에러를 검출하면 컨트롤러는 **재전송(retry)**한다. 영구(hard) 에러와 일시(soft) 에러를 구분하는 것 — CE가 같은 주소에서 반복되면 영구, 흩어지면 일시 — 도 RAS 정책의 일부다.

8.7 availability — degraded mode와 RowHammer 완화

degraded mode(성능 저하 운영): 일부가 고장나도 멈추지 않고 줄어든 능력으로 계속 간다. 예 — 한 채널/rank에 문제가 생기면 그것을 빼고 나머지로 운영(용량·대역폭은 줄지만 다운은 없음), 특정 bank/row를 **post-package repair(PPR)**로 여분 자원에 매핑해 격리(02 §8의 redundancy/repair를 필드에서 발동), memory mirroring(중요 영역을 이중화) 등.

RowHammer 완화 — 컨트롤러 레벨 TRR/RFM(08 §6과 연결): 08 §6에서 RowHammer를 "특정 row 반복 activate → 인접 row bit flip → 신뢰성·보안 위협"으로 배웠다. 그 완화의 **컨트롤러 측 책임**을 RAS 관점에서 정리한다.

- **RFM(Refresh Management)**: 컨트롤러가 각 bank의 **activation 횟수**를 추적하다가, 누적량이 임계(RAA, Rolling Accumulated ACT 등)를 넘으면 DRAM에 **RFM 명령**을 보내 추가 refresh 기회를 준다. 즉 컨트롤러가 "이 bank가 너무 두드러졌다"를 세서 미리 막는다.
- **TRR(Target Row Refresh)**: aggressor row를 감지해 인접 victim row를 추가 refresh. 구현이 on-die(DRAM 내부)일 수도, 컨트롤러 협력일 수도 있다(세대·벤더별 발전).

이것은 §5의 refresh 관리, §8의 에러 로깅과 한 가족이다 — **access 패턴을 카운트하고, 임계와 비교하고, 추가 명령(refresh)을 스케줄에 끼워 넣는** 디지털 제어 로직.

설계 브릿지(RAS 종합): §8 전체를 관통하는 사실 — ECC(인코더/디코더 = XOR 트리), scrubbing(background 순회 FSM + arbitration), error logging(카운터 + 임계 비교 + 인터럽트),

poison(태그 전파), retry(재전송 FSM), RFM(activation 카운터) 은 예외 없이 카운터·비교기·FSM·XOR 로직 + 타이밍 검증으로 구현된다. 08 §10의 "메모리 소자는 겹이지만 그 소자를 관리하는 로직은 100% 내 영역"이 RAS에서 정점을 찍는다. RAS는 메모리 시스템에서 가장 순수하게 디지털 설계·검증인 영역이고, 그래서 JD가 별도 키워드로 강조하며, 그래서 정확히 지원자의 무대다.

9. NAND/HBM 변주 — 같은 두뇌, 다른 미디어

9.1 NAND 미디어 컨트롤러 — DRAM 컨트롤러와 무엇이 다른가

JD의 "Media Controller(NAND, DRAM)"는 컨트롤러가 다루는 미디어가 DRAM만이 아님을 뜻한다. NAND를 다루는 컨트롤러(SSD 컨트롤러)는 DRAM 컨트롤러와 골격은 같지만(요청→큐→스케줄→미디어 명령), 미디어 특성이 달라 추가 계층이 두껍다.

03에서 본 NAND의 특성이 컨트롤러 설계를 바꾼다.

- **덮어쓰기 불가 + erase 단위 ≫ write 단위**: NAND는 페이지 단위로 쓰지만 지우기는 블록 단위, 그리고 같은 자리에 바로 못 덮어쓴다(먼저 지워야). 그래서 **FTL(Flash Translation Layer)**이 필요하다 — 호스트의 논리 주소(LBA)를 NAND의 물리 주소로 **매핑**하고, 덮어쓰기를 새 자리에 쓰고 옛 자리는 **무효화**한다. DRAM 컨트롤러의 단순한 주소 디코드(§6)와 달리, FTL은 **동적 매핑 테이블**을 관리하는 무거운 계층이다.
- **GC(Garbage Collection)**: 무효화된 페이지가 흩어진 블록들을 모아, 유효 데이터만 새 블록에 옮기고 옛 블록을 지워 공간을 회수한다. GC가 일반 트래픽과 경쟁하는 것은 §5의 refresh arbitration, §8의 scrubbing과 같은 "background 작업 vs 전경 트래픽" 구조다.
- **wear-leveling**(03 §7.4): P/E 수명이 유한하므로, 쓰기를 전 블록에 골고루 분산해 특정 블록만 빨리 늙는 것을 막는다.
- **강한 ECC**: NAND는 raw bit error rate가 DRAM보다 훨씬 높아, SECDED 수준이 아니라 **LDPC 같은 강력한 ECC**가 필수다(03 연결).

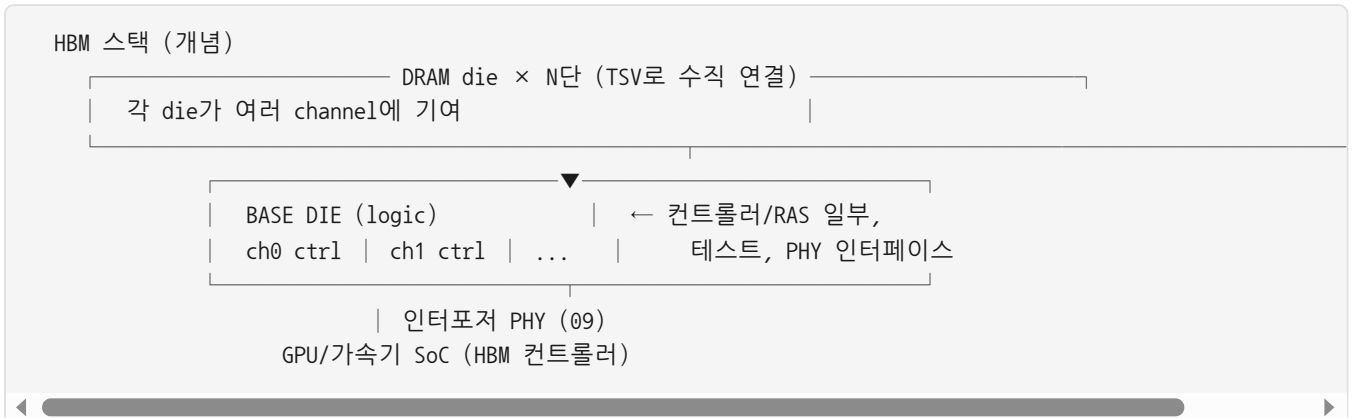
DRAM 컨트롤러	vs	NAND 미디어 컨트롤러(SSD)
주소 디코드(고정 매핑)		FTL(동적 매핑 테이블, LBA↔PBA)
refresh 스케줄		GC + wear-leveling 스케줄
on-die/system ECC		강한 ECC(LDPC) + read reclaim
타이밍 카운터(ns 단위)		NAND 명령 타이밍(μs 단위, 훨씬 느림)
→ 공통: 요청 큐 + 스케줄러 + 미디어 명령 FSM + ECC + background 작업 arbitration		

핵심: **NAND 컨트롤러는 DRAM 컨트롤러 위에 FTL·GC·wear-leveling이라는 두꺼운 관리 계층을 얹은 것이다.** 그 계층은 보통 **펌웨어(FW) + 하드웨어 가속 IP**로 구현된다(JD의 "HW IP 혹은 FW"가 여기서도). 그래도 밑바탕 — 큐, 스케줄러, 미디어 명령 FSM, ECC, background arbitration — 은 DRAM과 같은 디지털 제어 로직이다.

9.2 HBM의 다채널/pseudo-channel 구조에서 컨트롤러

HBM은 하나의 스택 안에 여러 독립 channel을 둔다(예: HBM2/2E는 스택당 8 channel, HBM3/3E는 그것을 다시 **pseudo-channel**로 쪼갬다 — 세대별 변동). 이 구조가 컨트롤러에 주는 의미:

- **채널 병렬성 극대화**: 각 channel이 자기 데이터 버스·명령을 독립적으로 가지므로, 컨트롤러는 여러 channel을 완전히 병렬로 운용한다(\$6 채널 인터리빙의 극단). 컨트롤러는 사실상 **여러 개의 독립 채널 컨트롤러 인스턴스**를 두고, 각자 \$2~\$5를 돌린다.
- **pseudo-channel**: 한 channel을 두 개의 절반(pseudo-channel)으로 나눠, 같은 명령 버스를 공유하되 데이터는 독립적으로 다룬다. 더 작은 granularity로 접근해 효율을 높이는데, 컨트롤러는 이 pseudo-channel의 명령/데이터 정렬을 관리해야 한다.
- **base die 통합**: 09 \$9에서 본 대로 HBM base die가 점점 똑똑해지며, 컨트롤러·RAS 기능의 일부가 base die로 내려갈 수 있다. SK하이닉스 JD의 "HBM SoC/RTL Design"이 가리키는 — base die에 컨트롤러·RAS·테스트 로직을 RTL로 설계하는 — 영역이다.



핵심: 채널이 많아지고 base die가 똑똑해질수록, 컨트롤러는 "여러 독립 채널 엔진 + 스택 차원 RAS + 테스트"를 통합하는 디지털 SoC가 된다. 이것이 HBM 시대에 컨트롤러·base die RTL 설계 수요가 커지는 이유다.

10. 검증 관점 — "메모리 검증"의 핵심은 컨트롤러다

10.1 컨트롤러 검증이 곧 "메모리 검증"인 이유

면접에서 "메모리 검증을 해봤나"는 질문이 들어오면, 핵심은 이것이다 — 메모리 시스템에서 검증의 무게중심은 셀 (아날로그)이 아니라 컨트롤러(디지털 프로토콜·로직)에 있다. 셀의 retention·sensing은 소자·공정·DFT(MBIST, 07 참조)가 보지만, "명령 시퀀스가 JEDEC 프로토콜을 지키는가, 타이밍을 어기지 않는가, 코너에서 데드락·데이터 깨짐이 없는가"는 RTL 검증의 영역이고, 그게 곧 지원자가 하던 일이다.

10.2 JEDEC 프로토콜 준수 — SVA/formal

컨트롤러가 발행하는 명령 시퀀스는 JEDEC 표준 상태도와 타이밍을 한 치도 어기면 안 된다. 이것을 검증하는 1순위 무기가 SVA(SystemVerilog Assertion)와 formal verification이다(07 연결).

프로토콜 assertion 예시 (개념)

- assert: ACT 후 tRCD 안 지나면 같은 bank에 RD/WR 금지
- assert: PRE 후 tRP 안 지나면 같은 bank에 ACT 금지
- assert: ACT 후 tRAS 안 지나면 PRE 금지
- assert: 임의 4개 ACT가 tFAW 윈도우보다 촘촘하면 금지
- assert: 닫힌 bank에 RD/WR 금지 (상태 위반)
- assert: REF 중인 bank에 일반 명령 금지

이런 assertion을 컨트롤러 출력(명령 버스)에 붙여, 모든 가능한 시나리오에서 위반이 없음을 formal로 증명하거나 시뮬레이션에서 감시한다. formal은 "반례가 존재하지 않음"을 수학적으로 보장하므로, 타이밍 규칙 위반 같은 절대 일어나선 안 되는 사건의 검증에 특히 강하다. 이것은 지원자가 삼성에서 프로토콜 체커·formal property로 하던 일과 정확히 같다.

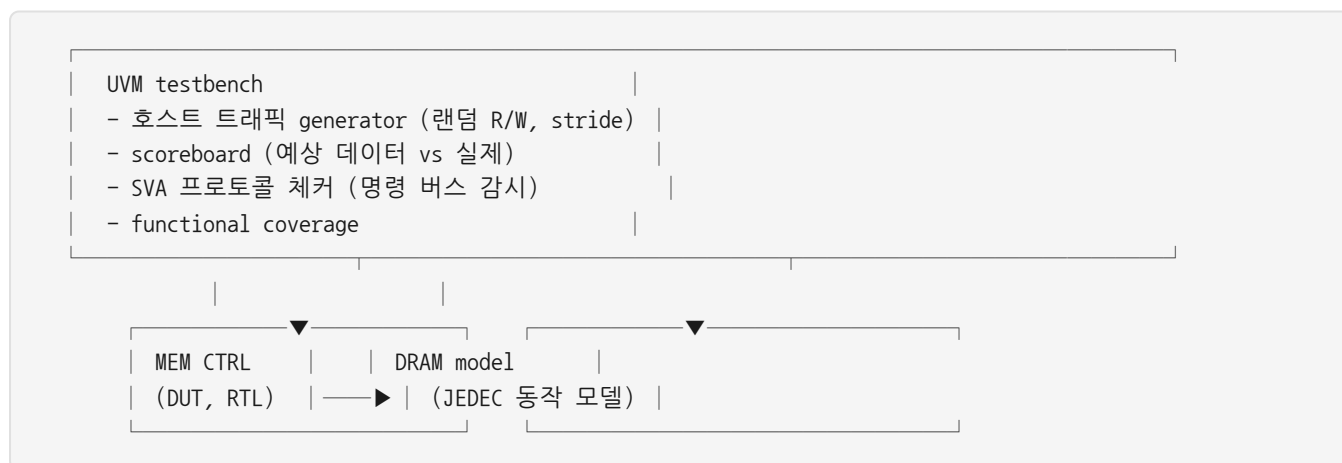
10.3 코너 케이스 — UVM CRV로 (07 연결)

프로토콜 합법성(SVA)이 "규칙을 안 어기나"라면, 기능 정확성·성능·데드락은 UVM 기반 제약 랜덤 검증(CRV, Constrained Random Verification)으로 본다(07 참조). 컨트롤러의 무서운 코너들:

- **refresh 충돌**: refresh가 임박한 순간에 같은 bank로 트래픽이 몰리면? postpone 한도와 retention 한도가 동시에 압박할 때 arbiter가 올바르게 refresh를 우선하는가? refresh 빛을 갚는 동안 데드락은 없는가?
- **turnaround 경계**: read→write→read가 최소 간격으로 빗발칠 때 tWTR/tRTW를 정확히 지키는가? 버스 방향 전환이 데이터 충돌(버스 contention) 없이 이뤄지는가?
- **outstanding 요청 한계**: 큐가 가득 찰 때(많은 미완료 요청), 새 요청 차단·기아 방지·순서 보존이 올바른가? write buffer가 꽉 찬 채 같은 주소 읽기(hazard)가 오면 forwarding이 정확한가?
- **저전력 진입/탈출 중 요청**: self-refresh로 들어가는 중에 새 요청이 오면? 탈출 시퀀스 도중의 명령 처리가 올바른가? (도메인 경계라 CDC도 함께.)
- **주소 매핑 충돌**: 특정 stride 접근이 bank conflict를 유발하는 pathological 패턴에서 성능 저하·정확성 문제는 없는가?

CRV는 이런 시나리오를 제약 하에 랜덤하게 대량 생성해, 사람이 미처 생각 못 한 조합까지 두드린다. 커버리지(functional coverage)로 "모든 명령 전이, 모든 타이밍 경계, 모든 큐 상태를 다 쳤나"를 측정한다 — 07에서 본 그 방법론 그대로.

10.4 검증 환경의 구성



호스트 트래픽을 랜덤 생성해 컨트롤러에 넣고, 컨트롤러가 내보내는 명령을 JEDEC DRAM 모델(표준 동작·타이밍을 모사)에 연결한다. scoreboard가 "쓴 데이터를 나중에 읽으면 같은가(데이터 무결성)"를 검사하고, SVA가 명령 버스의 프로토콜·타이밍 위반을 잡고, coverage가 검증 완전성을 측정한다. 이것이 메모리 컨트롤러 검증의 표준 환경이고, 삼성에서 IP 검증으로 구축하던 UVM 환경과 구조가 동일하다.

설계 브릿지(검증 종합): "프로토콜 준수 = SVA/formal, 코너 = UVM CRV, 데이터 무결성 = scoreboard, 완전성 = coverage" — 메모리 컨트롤러 검증의 4축이 전부 지원자의 검증 sign-off 역량과 1:1로 겹친다.

11. 면접 연결 — 30초 / 1분 골격 + 강점 브릿지

각 질문에 30초(핵심) → 1분(확장) → 브릿지 한 문장으로 준비한다.

Q. "메모리 컨트롤러가 하는 일은?"

- **30초:** "호스트의 추상적 read/write 요청을 받아, DRAM이 알아듣는 저수준 명령 시퀀스 — ACT로 행 열고, RD/WR로 열 접근하고, PRE로 닫고, REF로 refresh — 로 번역하는 **번역가이자 교통경찰**입니다. 주소를 bank/row/column으로 디코드하고, JEDEC 타이밍 규칙(tRCD/tRP/tRAS 등)을 어기지 않게 명령을 발행하고, 충돌하는 요청을 스케줄링하고, refresh를 끼워 넣고, ECC로 에러를 다룹니다. 셀은 멍청하고 똑똑함은 전부 컨트롤러에 있습니다."
- **1분 확장:** 위 + 명령/주소 경로(§2), 타이밍 집행(§3), 스케줄러가 대역폭·레이턴시를 좌우(§4), refresh arbitration(§5)을 한 사슬로. "이 전부가 FSM·큐·arbiter·카운터로 구현되는 디지털 제어 로직"임을 강조.
- **브릿지:** "추상 트랜잭션을 받아 프로토콜 FSM으로 하위 디바이스에 명령을 내리는 건 제가 삼성에서 다루던 AXI master·캐시·DMA 컨트롤러와 같은 패턴입니다 — 메모리 컨트롤러는 메모리 시스템에서 가장 RTL스러운 블록이고, 제 홈그라운드입니다."

Q. "open-page와 closed-page 정책의 차이는?"

- **30초:** "RD/WR 후 그 row를 **열어두느냐(open) 바로 닫느냐(closed)**의 선택입니다. open-page는 '근처를 또 접근할 것이다'라는 **지역성**에 베팅해, 같은 row 재접근(row hit)이면 ACT를 건너뛰어 빠릅니다. 단 다른 row가 오면(conflict) PRE 후 ACT라 오히려 느립니다. closed-page는 '다음은 다른 row일 것이다'에 베팅해 항상 닫아두니, conflict 페널티가 없고 latency가 일관됩니다. CPU처럼 지역성 높으면 open, 무작위 접근(그래픽·서버 mix)이면 closed가 유리합니다."
- **1분 확장:** row buffer hit/miss/conflict 개념, 적응형 정책(hit를 모니터링으로 전환), timeout 기반 절충까지.
- **브릿지:** "열어들지 닫을지를 런타임 통계로 판단하는 정책 로직은 캐시 교체 정책 같은 제어 블록이고, 그 정확성·성능을 검증하는 게 제 일입니다."

Q. "FR-FCFS가 뭐가?"

- **30초:** "메모리 스케줄러 알고리즘으로, **First-Ready(지금 즉시 발행 가능한 — 특히 열린 row를 때리는 row hit 요청)**를 먼저, 준비도가 같으면 **First-Come-First-Served(먼저 온 것)**로 처리합니다. 열린 row를 몰아서 우려먹어 bank 효율과 대역폭을 끌어올립니다. 단 특정 스트림이 다른 요청을 굶길 수 있어, 나이 기반 상한과 QoS로 보강합니다."
- **1분 확장:** 왜 순서 재배열이 성능을 좌우하나(§4.1), 타이밍 게이트 통과 후보 중 선택(§3+§4), bank/BG interleaving·turnaround batching과 함께 동작.
- **브릿지:** "준비도·나이·우선순위·버스 방향을 종합하는 다차원 arbiter라, 제가 짜던 arbitration 로직 그대로입니다."

Q. "RAS란? ECC 계층은?"

- **30초:** "RAS는 Reliability(에러를 정정해 답을 지킴)·Availability(에러가 나도 안 멈춤)·Serviceability(무슨 에러인지 기록·복구)입니다. ECC는 계층적입니다 — **on-die ECC**가 칩 내부 single-bit를, **link ECC**가 전송 에러를, **system ECC(chipkill)**가 칩 단위 fail까지 막습니다. 정정 가능 에러(CE)는 카운트해 예측적 교체에 쓰고, 정정 불가(UE)는 poison으로 격리합니다. AI 서버는 메모리가 TB급이고 수주씩 도니, 에러가 나도 살아남는 RAS가 필수입니다."
- **1분 확장:** scrubbing(patrol/demand로 1비트일 때 닦아 2비트로 키우지 않기), error logging(CE/UE 카운터 + 임계 경고), RowHammer 완화(컨트롤러 RFM, activation 카운트), HW/FW 분담.
- **브릿지:** "ECC 인코더/디코더는 XOR 트리, scrubbing은 background FSM, 로깅은 카운터+비교기, RFM은 activation 카운터 — RAS는 메모리에서 가장 순수하게 디지털 설계·검증인 영역이라, 정확히 제 일입니다."

Q. "scrubbing이 왜 필요한가?"

- **30초:** "오래 안 읽히는 메모리에 single-bit 에러가 하나 생기면, 아무도 안 읽으니 정정도 안 됩니다. 그러다 두 번째 비트까지 깨지면 SECEDED로는 정정 불가가 됩니다 — **고칠 수 있던 1비트를 방치해 못 고치는 2비트로 키우는** 사고죠. scrubbing은 주기적으로 메모리를 읽어 ECC로 정정·재기록해, 1비트일 때 닦아냅니다. 한가할 때 전체를 순찰하는 patrol scrub과, 읽다가 발견하면 고치는 demand scrub이 있습니다."
- **1분 확장:** refresh(전하 되채움, physical)와 scrubbing(비트 정정, logical)의 차이, 둘 다 background arbitration, AI 장시간 실행에서의 중요성.
- **브릿지:** "background 순회 + 일반 트래픽과의 arbitration + ECC 재기록은 refresh 스케줄링과 같은 제어 구조라, FSM·arbiter로 구현·검증하던 제 영역입니다."

통합 브릿지 — 외워둘 한 문장

"메모리 컨트롤러는 메모리 시스템의 디지털 두뇌입니다 — 추상 요청을 JEDEC 명령으로 번역하는 FSM, 대역폭을 짜내는 스케줄러, refresh와 ECC·scrubbing·RAS를 관리하는 제어 로직. 셀의 물리는 제가 새로 배울 갭이지만, **그 셀을 부리는 컨트롤러의 프로토콜 FSM·arbitration·타이밍 집행·SVA 프로토콜 검증·UVM 코너 검증**은 제가 4년간 RTL 설계·sign-off로 하던 일과 같은 문제입니다. 그래서 메모리 컨트롤러·RAS는 제게 가장 자연스러운 무대입니다."

12. 스스로 점검 — 다음 질문에 막힘없이 답할 수 있는가

1. **호스트 ↔ 컨트롤러 ↔ PHY ↔ DRAM die**의 4단 사슬에서 각자의 책임을 말하고, 컨트롤러가 물리적으로 어디에 있는지(DDR/HBM/NAND 각각)를 설명할 수 있는가?
2. 호스트의 READ 하나가 왜 ACT/RD/PRE **여러 명령**으로 번역되는지를 02의 DRAM 물리(행 열고-읽고-닫기)로부터 설명하고, row hit과 row miss의 명령 차이를 말할 수 있는가?
3. **tRCD/tRP/tRAS/tWR/tCCD/tRRD/tFAW**를 "컨트롤러가 어느 두 명령 사이에 강제하는 최소 간격"으로 각각 설명하고, 특히 tRRD/tFAW가 **전력 제약**에서 나온 이유를 말할 수 있는가?
4. 컨트롤러가 타이밍을 **down-counter 배열**로 추적하고, "게이트 통과 후보 중 정책으로 선택"하는 2단계 구조를 설명할 수 있는가? 이것이 STA의 setup 보장과 어떻게 정적/동적으로 대응되는가?

5. **open-page vs closed-page**를 지역성/충돌의 베틱으로 설명하고, 각각 유리한 워크로드와 적응형 정책의 동기를 말할 수 있는가?
6. **FR-FCFS**의 두 우선순위(ready, age)와 그것이 row hit을 몰아 bank 효율을 올리는 원리, 그리고 기아(starvation) 문제와 보강책을 설명할 수 있는가?
7. **bank/BG interleaving**과 **read/write turnaround batching**, **write buffering/hazard forwarding**이 각각 왜 대역폭·레이턴시를 개선하는지 말할 수 있는가?
8. **all-bank vs per-bank refresh**, refresh **postpone/pull-in**, 일반 트래픽과의 **deadline-aware arbitration**, self-refresh 진입/탈출 시퀀스를 설명할 수 있는가? 이것이 왜 "clock/reset & power control logic"인가?
9. **주소 매핑**이 왜 성능 결정인지를 "연속 주소를 다른 bank/channel로 흩어 병렬성을 만든다"로 설명하고, 좋은/나쁜 매핑의 차이와 XOR 해싱의 목적을 말할 수 있는가?
10. **컨트롤러-PHY 경계**에서 컨트롤러가 조율하는 것(training/leveling 지휘, ODT 제어, ZQ 트리거, DQS 정렬)과 PHY가 자체 처리하는 것을 구분하고, 다중 클럭/전원 도메인 제어가 왜 CDC/RDC/UPF 검증인지 말할 수 있는가?
11. **RAS의 R/A/S** 정의와, **ECC 3계층**(on-die/link/system-chipkill)이 각각 막는 에러를 구분하며, 왜 AI/데이터센터에서 RAS가 필수인지(규모×시간, SDC) 설명할 수 있는가?
12. **scrubbing**(patrol/demand)이 왜 필요한지(1비트→2비트 방치 사고)를 설명하고, refresh와의 차이(physical vs logical)를 말할 수 있는가?
13. **CE/UE, poison, retry, degraded mode**, **컨트롤러 RFM(RowHammer)**를 각각 "Reliability/Availability/Serviceability 중 무엇에 기여하나"로 분류해 설명할 수 있는가?
14. **NAND 미디어 컨트롤러**가 DRAM 컨트롤러와 다른 점(FTL·GC·wear-leveling·강한 ECC)과, **HBM의 다채널/pseudo-channel**에서 컨트롤러가 어떻게 확장되는지 말할 수 있는가?
15. **컨트롤러 검증**의 4축(SVA/formal 프로토콜 준수, UVM CRV 코너, scoreboard 데이터 무결성, coverage 완전성)을 설명하고, 대표 코너(refresh 충돌·turnaround·outstanding·저전력 전환)를 들 수 있는가?
16. (종합) "셀은 갭이지만 컨트롤러는 내 영역"이라는 명제를, **프로토콜 FSM·스케줄러 arbitration·타이밍 집행·RAS 제어 로직·SVA/UVM 검증** 다섯 사례로 즉석에서 뒷받침할 수 있는가?

⚠ 정확성 노트: 본문의 구체 수치·구조($t_{REFI} \approx 7.8\mu s$, postpone 최대 횟수, HBM channel/pseudo-channel 수, scrub 주기, ECC 코드 강도 등)는 **세대·표준 버전·제품·구성별로 변동**한다. 면접에서는 "원리와 방향성"(예: "open-page는 지역성에 베틱한다", "scrubbing은 1비트를 2비트로 키우지 않게 미리 닦는다", "FR-FCFS는 row hit을 우선해 bank 효율을 올린다")을 정확히 말하고, 구체 숫자는 "약/급/변동" 단서를 붙여 단정하지 말 것. 내부 비공개 구현 세부는 날조하지 말고, 공개 JEDEC 표준·아키텍처 원리 수준에서 설명할 것. 세부 회로보다 **"디지털 설계자가 이해하고 검증해야 할 제어 로직·프로토콜·RAS 정책"**을 우선한다.